Task 04 - What Is the Orchestration Layer?

Handbook reference: Page 4 of 10 | Report date: 2026-10-13 | Folder: `task-04-orchestration-layer/`

1. Concept from the handbook

The orchestration layer is the application logic around the model: task routing, context assembly, permissions, reliability (retry, timeout, checkpoints, logging) and completion logic. A smart prompt alone cannot grant file access, enforce permissions, persist state or guarantee tool execution. Even a single agent needs an orchestrator.

2. What was built

An **Orchestrator** with a tool registry, a **Permissions** object (allow-list, write scope, human-approval tools), **bounded retries only for transient errors**, per-call timeout, JSON **checkpoints** with resume, a JSONL **audit log**, and completion / escalation logic.

3. Behaviours demonstrated

- Permission is checked BEFORE a tool runs
- Only TransientError is retried; real bugs surface immediately
- Timeout and exhausted retries -> ESCALATE
- Checkpoint resume skips already-done steps
- Production deploy requires human approval

4. Files in this task

`demo.py`, `orchestrator.py`, `test_orchestrator.py`

Run it: `python demo.py` Test it: `python -m unittest discover`

5. Code screenshots

orchestrator.py

```python
"""Task 04 - Orchestration layer: the control plane around the model.

Jobs implemented here (from the handbook): task routing, context assembly,
permissions, reliability (retry / timeout / checkpoint / logging) and
completion / escalation. A prompt alone cannot do any of these.
"""
from __future__ import annotations

import json
import time
from concurrent.futures import ThreadPoolExecutor, TimeoutError as FutTimeout
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any, Callable, Dict, List, Optional


class TransientError(Exception):
    """Failure worth retrying (network blip, lock...)."""


class PermissionDenied(Exception):
    pass


class Escalate(Exception):
    """Human authority/information needed."""


@dataclass
class Permissions:
    allowed_tools: set
    write_roots: List[str] = field(default_factory=list)   # directories tools may write in
    needs_human: set = field(default_factory=set)          # e.g. {"deploy_prod"}

    def check(self, tool: str, path: Optional[str] = None) -> None:
        if tool in self.needs_human:
            raise Escalate(f"'{tool}' requires human approval")
        if tool not in self.allowed_tools:
            raise PermissionDenied(f"tool '{tool}' not allowed")
        if path is not None and not any(path.startswith(r) for r in self.write_roots):
            raise PermissionDenied(f"path '{path}' outside write scope {self.write_roots}")


@dataclass
class Orchestrator:
    tools: Dict[str, Callable[..., Any]]
    permissions: Permissions
    audit_path: Path
    state_path: Path
    max_retries: int = 2
    timeout_s: float = 5.0
    backoff_s: float = 0.01
    state: Dict[str, Any] = field(default_factory=lambda: {"done": [], "results": {}})

    # ---------- logging / checkpoints ----------
    def _log(self, **event) -> None:
        event["t"] = round(time.time(), 3)
        with self.audit_path.open("a") as f:
            f.write(json.dumps(event, default=str) + "\n")

    def checkpoint(self) -> None:
        self.state_path.write_text(json.dumps(self.state, indent=2, default=str))

    @classmethod
    def resume_state(cls, state_path: Path) -> Dict[str, Any]:
        return json.loads(Path(state_path).read_text())

    # ---------- executing one step ----------
    def call(self, step_id: str, tool: str, **args) -> Any:
        self.permissions.check(tool, args.get("path"))        # permission BEFORE execution
        attempt = 0
        while True:
            attempt += 1
            try:
                with ThreadPoolExecutor(max_workers=1) as ex:
                    result = ex.submit(self.tools[tool], **args).result(timeout=self.timeout_s)
                self._log(step=step_id, tool=tool, attempt=attempt, status="ok")
                return result
            except TransientError as exc:                     # only transient errors retry
                self._log(step=step_id, tool=tool, attempt=attempt, status="transient", error=str(exc))
```

Main implementation: orchestrator.py (part 1/2)

```
                if attempt > self.max_retries:
                    raise Escalate(f"{tool} still failing after {attempt} attempts: {exc}")
                time.sleep(self.backoff_s * attempt)
            except FutTimeout:
                self._log(step=step_id, tool=tool, attempt=attempt, status="timeout")
                raise Escalate(f"{tool} timed out after {self.timeout_s}s")

    # ---------- the plan runner (routing + completion logic) ----------
    def run(self, plan: List[dict], is_complete: Callable[[Dict[str, Any]], bool]) -> dict:
        for step in plan:
            sid = step["id"]
            if sid in self.state["done"]:                       # resume after crash
                self._log(step=sid, status="skipped (checkpoint)")
                continue
            try:
                res = self.call(sid, step["tool"], **step.get("args", {}))
            except (PermissionDenied, Escalate) as exc:
                kind = "ESCALATE" if isinstance(exc, Escalate) else "DENIED"
                self._log(step=sid, status=kind, reason=str(exc))
                self.checkpoint()
                return {"status": kind, "step": sid, "reason": str(exc)}
            self.state["results"][sid] = res
            self.state["done"].append(sid)
            self.checkpoint()
        return {"status": "COMPLETE" if is_complete(self.state) else "INCOMPLETE",
                "done": list(self.state["done"])}
```

Main implementation: orchestrator.py (part 2/2)

demo.py

```python
import json, tempfile
from pathlib import Path
from orchestrator import Orchestrator, Permissions, TransientError

work = Path(tempfile.mkdtemp())
attempts = {"n": 0}

def read_tests():            return "3 existing tests found"
def flaky_ci_status():
    attempts["n"] += 1
    if attempts["n"] < 3:
        raise TransientError("CI API 503")
    return "CI reachable"
def write_file(path, text):  return f"wrote {path}"
def deploy_prod():           return "deployed"

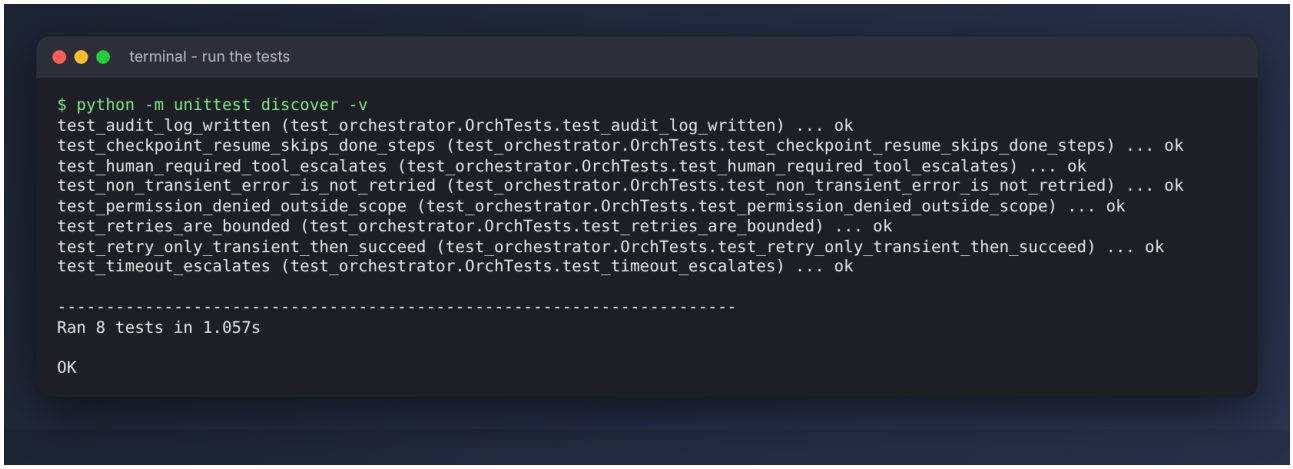
tools = {"read_tests": read_tests, "ci_status": flaky_ci_status, "write_file": write_file, "deploy_prod": deploy_prod}
perms = Permissions(allowed_tools=set(tools), write_roots=["src/", "tests/"], needs_human={"deploy_prod"})

def run(title, plan, **kw):
    o = Orchestrator(tools, perms, work / "audit.jsonl", work / f"state_{title}.json", **kw)
    result = o.run(plan, lambda s: len(s["done"]) == len(plan))
    print(f"{title:<28} -> {result}")

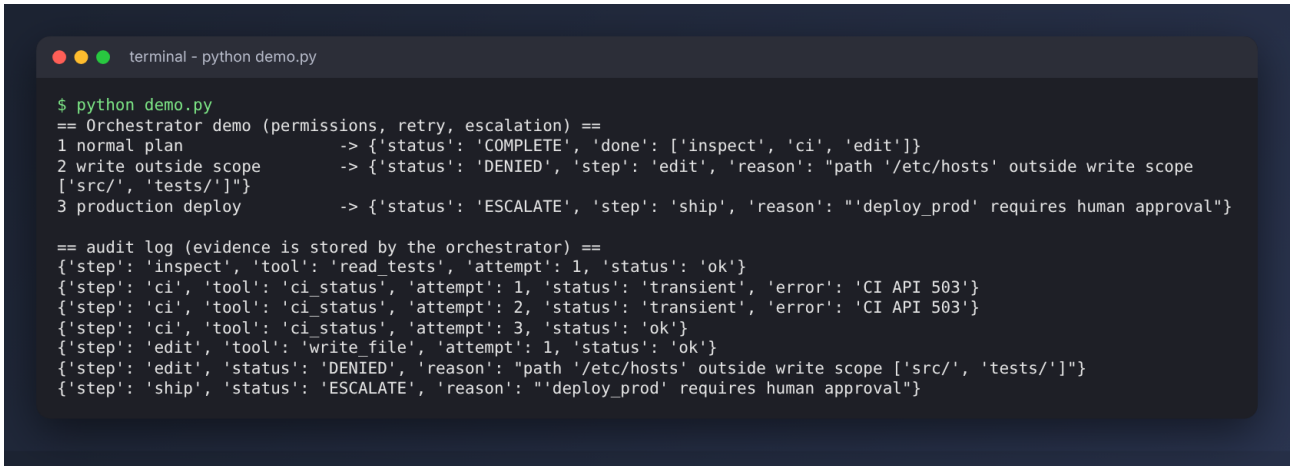
print("== Orchestrator demo (permissions, retry, escalation) ==")
run("1 normal plan", [{"id": "inspect", "tool": "read_tests"},
                      {"id": "ci", "tool": "ci_status"},
                      {"id": "edit", "tool": "write_file", "args": {"path": "src/tracker.py", "text": "..."}}])
run("2 write outside scope", [{"id": "edit", "tool": "write_file", "args": {"path": "/etc/hosts", "text": "x"}}])
run("3 production deploy", [{"id": "ship", "tool": "deploy_prod"}])
print("\n== audit log (evidence is stored by the orchestrator) ==")
for line in (work / "audit.jsonl").read_text().splitlines():
    e = json.loads(line); e.pop("t")
    print(e)
```

Demo script

6. Demo and test screenshots



Test run (real output)



Demo output (python demo.py)

7. Evidence-backed report

Change	What Is the Orchestration Layer?
Revision tested	content hash `af2b4677c6e0` of all .py files in this folder (re-run with the git commit SHA after you commit)
Checks run	`python -m unittest discover` and `python demo.py`
Results	**PASS** - Ran 8 tests in 1.058s
Checks not run	Concurrency / multi-process locking
Known limitations	Timeout uses a thread; a stuck tool thread cannot be force-killed in Python.
Next required action	Human review, then CI on the pull request

8. Practical takeaway

The model is the proposer; the orchestration layer is the control plane. Keep them conceptually separate.